CS5491 AI · Module B: Reinforcement Learning
RL 基础 · Model-Based/Free · TD Learning · Q-Learning · Approx. Q · Exploration

0. 翻页索引 Navigation

想找什么	位置
RL vs MDP 关系	P1 §1
Model-based vs free	P1 §2
Passive vs Active RL	P1 §3
TD learning 公式 + 更新	P1 §4
Q-learning 公式 + 更新	P1 §5
为什么学 Q 不学 V	P1 §5.1
Approx Q-learning	P2 §6
Tutorial: Car 例题	P2 §7
Exploration/ϵ-greedy	P2 §8
常见混淆 Quick Ref	P2 §9
简答题模板	P2 §10

1. 大局观 Big Picture

RL 一句话： Agent 和环境交互，收到 **reward**，学一个 **policy** 使长期期望回报最大。**不给正确答案，只给 reward signal。**

RL 与 MDP 的关系（必考）

MDP：环境模型已知 $T(s,a,s')$, $R(s,a,s')$，做 planning。
RL：仍假设 MDP 结构，但 **模型未知**，从样本中学策略。
RL assumes an MDP, but T and R are unknown and must be learned from experience.

MDP 基础速查

- States S, Actions A, $T(s,a,s')$, $R(s,a,s')$, γ, policy $\pi(s)$
- **Markov property**: 未来只依赖当前状态，不依赖历史。
  Future depends only on current state.
- **Return**: $U = r_0 + \gamma r_1 + \gamma^2 r_2 + \cdots$

Bellman 方程（必背）

$$V^*(s) = \max_a Q^*(s,a)$$

$$Q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

2. Model-Based vs Model-Free RL

Model-Based	Model-Free
先估计 $\hat{T}$, $\hat{R}$	不估计模型
再按 learned MDP 做 planning	直接从 sample 学 V 或 Q
样本效率高	简单直接
建模成本高	可能需要更多交互

Model-based: learn model first, then plan.
Model-free: directly update values from transitions.

Model-Based 流程

```
1. Count outcomes for each (s,a)
2. Normalize -> T_hat(s,a,s')
3. Average rewards -> R_hat(s,a,s')
4. Run value/policy iteration on learned MDP
```

3. Passive RL vs Active RL

Passive RL	Active RL
策略**固定**不能改	自己**选动作**
目标：评估该策略的 V^π	目标：学最优 policy / Q
方法：Direct Eval, TD	方法：Q-learning, ε-greedy

Passive: fixed policy, learn its values.
Active: choose actions, learn optimal policy.

Direct Evaluation（直接评估）

- 按固定策略执行，记每次访问某状态后的 observed return，取平均。
- **优点**：简单，不需知道 T, R，最终值正确。
- **缺点**：浪费状态间的联系信息，学得慢。

Estimates $V(s)$ by averaging observed returns from visits to s.

Direct Evaluation 做题模板

若某状态 s 被访问 m 次，后续折扣回报分别为 $G_1, \ldots, G_m$:

$$V^\pi(s) = \frac{G_1 + \cdots + G_m}{m}, \quad G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots$$

注意： 它用真实完整 return；TD 用 $r + \gamma V(s')$ 这种一跳估计。

4. TD Learning（时序差分，Passive RL）

核心思想

不等整条 trajectory 结束，**每一步更新**。用「当前奖励＋下一状态估值」拉近当前估值。
Update $V(s)$ toward sampled target after each step. No model needed.

Sample & Update（必背）

Experience: $(s, \pi(s), s', r)$
Sample target:

$$\text{sample} = r + \gamma V(s')$$

TD Update:

$$V(s) \leftarrow (1-\alpha)\, V(s) + \alpha \cdot \text{sample}$$

等价写法:

$$V(s) \leftarrow V(s) + \alpha \underbrace{(r + \gamma V(s') - V(s))}_{\text{TD error } \delta}$$

做题步骤 Step-by-Step

```
Given: (s, action, s', r), alpha, gamma, V table
Step 1: Compute sample = r + gamma * V(s')
Step 2: Compute TD error = sample - V(s)
Step 3: Update: V(s) <- V(s) + alpha * TD_error
Step 4: All other V(s) unchanged this step
```

Learning Rate α 的意义

- α 大：看重新样本，学快但不稳。
- α 小：更平滑稳定，学慢。
- 递减 α 常有助于收敛。

TD is an exponential moving average; recent samples get larger weight.

5. Q-Learning（Active RL，必考）

为什么要学 Q 而不只学 V？（必答）

若只有 $V(s)$，选动作还要算:

$$Q(s,a) = \sum_{s'} T(s,a,s')[R + \gamma V(s')]$$

但这需要 T, R！在 model-free 场景无法做到。
学 $Q(s,a)$ 可直接 arg max$_a$ $Q(s,a)$**，不需要模型。**
Q-values allow direct action selection without knowing T or R.

Q-learning Sample & Update（必背）

Experience: (s, a, s', r)
Sample target:

$$\text{sample} = r + \gamma \max_{a'} Q(s', a')$$

Q-learning Update:

$$Q(s,a) \leftarrow (1-\alpha)\, Q(s,a) + \alpha \cdot \text{sample}$$

等价写法:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big)$$

做题步骤 Step-by-Step

```
Given: (s, a, s', r), alpha, gamma, Q table
Step 1: Find all Q(s', a') for next state s'
Step 2: Compute max_{a'} Q(s', a')
Step 3: Compute sample = r + gamma * max Q(s',a')
Step 4: Update: Q(s,a) <- Q(s,a) + alpha*(sample - Q(s,a))
Step 5: Only Q(s,a) changes; rest unchanged
```

性质

- **Off-policy**：即使行为策略不是最优，仍可收敛。
- 条件：探索充分 + 学习率逐渐递减。

Q-learning is model-free and off-policy. It converges to Q^* given sufficient exploration.

Q-learning 易错细节

- 若 s' 是 terminal：通常令 max$_{a'}$ $Q(s',a') = 0$，target $= r$。
- 只更新当前经历里的 $Q(s,a)$，不是整行、不是 $Q(s',a')$。
- 计算 target 时用更新前的旧 Q 表；更新后再进入下一步。
- 若多个动作并列最大，policy 可任选；计算 max 数值不受影响。

TD vs Q-Learning 对比速查

	TD Learning	Q-Learning
学什么	$V(s)$（策略评估）	$Q(s,a)$（最优值）
类型	Passive RL	Active RL
动作选取	需要模型	直接 arg max$_a$ Q
核心目标	$(r + \gamma V(s'))$	$(r + \gamma \max_{a'} Q(s',a'))$

Value Iteration 快速回顾（MDP）

Value Iteration update:

$$V_{k+1}(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_k(s')]$$

Grid World 做题步骤

```
Given: grid, T(stochastic), R, gamma, V_0=0
For each state s (not terminal):
  For each action a:
    Compute sum_{s'} T(s,a,s') * [R(s,a,s') + gamma*V(s')]
  V_new(s) = max over all actions
Terminal states: V = terminal reward, fixed
```

典型 transition（stochastic 80/10/10）

Action=Up: P(Up)=0.8, P(Left)=0.1, P(Right)=0.1
若撞墙则停在原地（stay in place）。
一定要考虑所有可能的 s'，包括撞墙的情况！

必背公式总表 Formula Summary

Bellman:

$$V^*(s) = \max_a Q^*(s,a)$$

$$Q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

TD Learning:

$$\text{sample} = r + \gamma V(s')$$

$$V(s) \leftarrow V(s) + \alpha(r + \gamma V(s') - V(s))$$

Q-Learning:

$$\text{sample} = r + \gamma \max_{a'} Q(s',a')$$

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\Big)$$

Approx. Q-Learning:

$$Q(s,a) = \sum_i w_i f_i(s,a)$$

$$\Delta = (r + \gamma \max_{a'} Q(s',a')) - Q(s,a)$$

$$w_i \leftarrow w_i + \alpha\, \Delta\, f_i(s,a)$$

考试识别：看到题目怎么选公式

题目关键词	直接套
fixed policy / evaluate policy	Passive RL: Direct Eval 或 TD
one transition + V table	TD: $V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$
one transition + Q table	Q-learning: target $r + \gamma \max Q(s',a')$
features/weights/sensors	Approx Q: 先算 Δ，再改 w_i
unknown T, R but learn model	Model-based: count $\hat{T}$, average $\hat{R}$, then plan
random vs greedy action	ϵ-greedy / exploration tradeoff

计算题落笔顺序

先写 **target/sample**，再写 **error = target-current**，最后写 **new = old + αerror**。这样 TD、Q、Approx-Q 都不会乱。

Module B (Back): Approximate Q-Learning & Tutorial

Approx Q · Car Tutorial · Exploration · ε-greedy · Confusions · Templates

6. Approximate Q-Learning（近似 Q 学习）

为什么需要?

状态空间太大，无法为每个 (s, a) 存储一个 Q 值。需要 **泛化（generalization）**。

When the state space is huge, store Q as a parameterized function, not a table.

核心公式（必背）

Q 用特征线性近似:

$$Q(s,a) = w_1 f_1(s,a) + w_2 f_2(s,a) + \cdots + w_n f_n(s,a) = \sum_i w_i f_i(s,a)$$

TD 误差:

$$\Delta = \underbrace{\left(r + \gamma \max_{a'} Q(s',a')\right)}_{\text{target}} - \underbrace{Q(s,a)}_{\text{current}}$$

权重更新:

$$w_i \leftarrow w_i + \alpha\, \Delta\, f_i(s,a)$$

直觉

- $\Delta > 0$: 当前 Q 估低了 ⇒ 提升相关特征权重。
- $\Delta < 0$: 当前 Q 估高了 ⇒ 压低相关特征权重。
- 更新一次 w_i 会影响**所有相似状态**的 Q 值（泛化）。

做题步骤 Step-by-Step

```
Given: weights w, features f, transition (s,a,r,s'), alpha, gamma
Step 1: Compute Q(s,a) = sum_i w_i * f_i(s,a)
Step 2: For each action a' from s':
          Q(s',a') = sum_i w_i * f_i(s',a')
Step 3: Compute max_{a'} Q(s',a')
Step 4: Compute Delta = (r + gamma*max Q(s',a')) - Q(s,a)
Step 5: For each feature i:
          w_i <- w_i + alpha * Delta * f_i(s,a)
Note: Only features of CURRENT (s,a) are updated!
      f_i(s',a') does NOT appear in weight update.
```

易错: 更新权重用 $f_i(s,a)$（当前状态动作的特征），**不是** $f_i(s',a')$!

Approx Q 符号拆解

- $f_i(s,a)$: 题目给的特征值；没选的 action 特征通常为 0。
- w_i: 特征的重要性；正权重鼓励该特征，负权重惩罚该特征。
- Δ: 这一笔经验认为当前 $Q(s,a)$ 低估/高估多少。
- w_i 更新后，所有用到该特征的未来 Q 值都会变，这就是 generalization。

Approximate Q-learning updates weights, not a single table entry.

Approx Q 答题模板

```
1. Current action features: write f_i(s,a)
2. Current Q: Q(s,a)=sum_i w_i f_i(s,a)
3. Next state:
   compute Q(s',A), Q(s',B), ... using SAME weights
4. Target = r + gamma * max_next_Q
5. Delta = target - current_Q
6. Update each weight:
   w_i <- w_i + alpha * Delta * f_i(s,a)
7. State final weight vector clearly
```

检查: 若当前特征 $f_i(s,a) = 0$，该 w_i 本步不变；即使下一状态该特征非零，也不在本步更新。

7. Tutorial: Car Approx Q-Learning（逐步精算）

题目设定

Actions: Accelerate (A) / Brake (B)

Rewards: safe move=+1; no move=0; hit car=−2

$\gamma = 1,\ \alpha = 0.5$

特征（每个 action 两个特征）:

For A: f_{AD}(distance), f_{AS}(speed)

For B: f_{BD}(distance), f_{BS}(speed)

注意: 取 action=A 时，$f_{BD} = f_{BS} = 0$; 取 B 时，$f_{AD} = f_{AS} = 0$。

初始权重: $w_{AD} = 1,\ w_{AS} = 0,\ w_{BD} = 0,\ w_{BS} = 0$

Step 1 计算过程

数据: State s: $D = 0, S = 2$; Action: **A**; Reward: $r = -2$; Next s': $D = 1, S = 0$

当前特征（action=A）: $f_{AD} = 0,\ f_{AS} = 2,\ f_{BD} = 0,\ f_{BS} = 0$

(1) 计算当前 $Q(s,A)$:

$$Q(s,A) = 1\cdot 0 + 0\cdot 2 + 0\cdot 0 + 0\cdot 0 = 0$$

(2) 计算下一状态 $s' = (D = 1, S = 0)$ 的 Q 值:

$$Q(s',A) = w_{AD}\cdot 1 + w_{AS}\cdot 0 = 1\cdot 1 + 0\cdot 0 = 1$$

$$Q(s',B) = w_{BD}\cdot 1 + w_{BS}\cdot 0 = 0\cdot 1 + 0\cdot 0 = 0$$

$$\max_{a'} Q(s',a') = \max(1,0) = 1$$

(3) 计算 Δ:

$$\Delta = (-2 + 1\cdot 1) - 0 = -1$$

(4) 更新权重（action=A 的特征）:

$$w_{AD} \leftarrow 1 + 0.5\cdot(-1)\cdot 0 = 1$$

$$w_{AS} \leftarrow 0 + 0.5\cdot(-1)\cdot 2 = -1$$

w_{BD}, w_{BS} 不变（不是当前 action 的特征）。

Step 1 后权重: $w_{AD} = 1,\ w_{AS} = -1,\ w_{BD} = 0,\ w_{BS} = 0$

Step 2 计算过程

数据: State s: $D = 1, S = 0$; Action: **B**; Reward: $r = 0$; Next s': $D = 1, S = 0$

当前特征（action=B）: $f_{AD} = 0,\ f_{AS} = 0,\ f_{BD} = 1,\ f_{BS} = 0$

(1) 计算当前 $Q(s,B)$:

$$Q(s,B) = 1\cdot 0 + (-1)\cdot 0 + 0\cdot 1 + 0\cdot 0 = 0$$

(2) 计算 $Q(s',A), Q(s',B)$（s'=s 这里）:

$$Q(s',A) = 1\cdot 1 + (-1)\cdot 0 = 1$$

$$Q(s',B) = 0\cdot 1 + 0\cdot 0 = 0$$

$$\max_{a'} Q(s',a') = 1$$

(3) 计算 Δ:

$$\Delta = (0 + 1\cdot 1) - 0 = 1$$

(4) 更新权重（action=B 的特征）:

$$w_{BD} \leftarrow 0 + 0.5\cdot 1\cdot 1 = 0.5$$

$$w_{BS} \leftarrow 0 + 0.5\cdot 1\cdot 0 = 0$$

Step 2 后权重: $w_{AD} = 1,\ w_{AS} = -1,\ w_{BD} = 0.5,\ w_{BS} = 0$

用 learned weights 选动作（$D = 1, S = 1$）

$$Q(s,A) = 1\cdot 1 + (-1)\cdot 1 + 0.5\cdot 0 + 0\cdot 0 = 0$$

$$Q(s,B) = 1\cdot 0 + (-1)\cdot 0 + 0.5\cdot 1 + 0\cdot 1 = 0.5$$

$$\Rightarrow Q(s,B) > Q(s,A) \Rightarrow \textbf{prefer action B}$$

Car 题变形怎么套

- 若 $\gamma \neq 1$: 只改 $\Delta = (r + \gamma \max Q(s',a')) - Q(s,a)$。
- 若 α 变: 只改最后 $w_i \leftarrow w_i + \alpha\Delta f_i(s,a)$ 的步长。
- 若初始 weights 变: 重新算每个 Q，不要沿用 tutorial 数字。
- 若 action=A: 只会更新 A 相关非零特征; action=B 同理。
- 最后问 prefer action: 分别算 $Q(s,A)$ 和 $Q(s,B)$，选大的。

Car 题检查清单

- 每次更新后立刻写新的 weight vector，下一步必须用新权重。
- $Q(s',a')$ 是为了找下一状态最好动作；它本身不直接更新权重。
- Reward 是当前 transition 的 reward，不是下一状态 Q，也不是最终 prefer action。
- 题目若给多个 observed data，要按时间顺序一条一条更新。
- prefer action 不再更新，只比较 learned Q-values。

8. Exploration vs Exploitation

核心矛盾

- **Exploration（探索）**: 尝试不确定动作，收集信息，发现更优策略。
- **Exploitation（利用）**: 按当前最优估计行动，获得即时高 reward。

只 exploit: 永远学不到更好动作。只 explore: 当前收益很差。

The tradeoff: more exploration may reduce short-term reward but improve long-term policy.

ε-Greedy（必背）

以概率 ε: 随机选动作 **(explore)**

以概率 $1 - \varepsilon$: 选 $\arg\max_a Q(s,a)$ **(exploit)**

- **优点**: 简单，保证持续探索。
- **缺点**: 学好后仍 random thrashing。
- 常见做法: 让 ε 随时间下降。

ε-greedy: random action with prob ε, greedy otherwise.

Exploration Function（了解）

对访问次数少的状态给 optimism bonus:

$$f(u,n) = u + \frac{k}{n+1}$$

n=visit count; n 小时 bonus 大; n 大后趋向 exploitation。

9. 常见混淆速判 Common Confusions

RL	MDP
模型**未知**，学习	模型**已知**，planning
Passive RL	**Active RL**
policy 固定，学 V^π	选动作，学最优 Q
TD Learning	**Q-Learning**
学 $V(s)$，policy eval	学 $Q(s,a)$，action sel.
Direct Eval.	**TD Learning**
等整条 return 平均	每步更新，更快
Model-Based	**Model-Free**
先学 $\hat{T}, \hat{R}$ 再算	直接学 V/Q

10. 简答题模板 Short Answer Templates

RL 与 MDP 的关系

RL assumes an MDP, but unlike MDP planning, the transition and reward functions are unknown and must be learned from experience.

为什么只学 V 不够

State values alone require the model for action selection. Q-values allow direct action selection via $\arg\max_a Q(s,a)$ without knowing T or R.

TD vs Model-Based

TD learning updates values from sampled transitions without estimating T or R. Model-based learning first estimates $\hat{T}$ and $\hat{R}$, then solves the learned MDP.

Passive vs Active RL

Passive RL follows a fixed policy to evaluate its values. Active RL must choose actions and learns an optimal policy.

ε-greedy 在做什么

It forces exploration by choosing random actions with probability ε, and exploits current knowledge by choosing the greedy action otherwise.

Q-learning 是什么

Q-learning is a model-free, off-policy algorithm that updates $Q(s,a)$ toward the target $r + \gamma \max_{a'} Q(s',a')$ from sampled transitions.

Approx Q-learning 是什么

Approximate Q-learning represents the action value as a linear combination of features. It updates weights using the TD error Δ, allowing generalization across states.

Direct Evaluation vs TD

Direct evaluation waits for complete observed returns and averages them. TD learning bootstraps from the next state's current value estimate, so it can update after every transition.

Exploration vs Exploitation

Exploration chooses uncertain actions to improve future knowledge; exploitation chooses the currently best action to get reward now. Active RL needs both.

Model-based 计算怎么写

Estimate $\hat{T}(s,a,s')$ by normalized transition counts and estimate $\hat{R}$ by averaging observed rewards, then solve the learned MDP using planning.

收敛/充分探索

With enough exploration and a suitable decreasing learning rate, Q-learning can converge to optimal Q-values even if the behavior policy is not always greedy.